

Normative Core

Actor Grounding Layer v0.1

AGL - Normative Core

Status	Draft v0.1
Layer	Source-state qualification / execution grounding / pre-review gate
Canonical home	ester-reality-bound
Author	Kotov Ivan
Location	Brussels
Year	2026

Kotov Ivan
Bruxelles, 2026

Contents

Abstract	2
1. Purpose	2
2. Scope	2
3. Non-goals	3
4. Core principle	3
5. Architectural position	4
6. What AGL qualifies	5
7. Grounding dimensions	6
8. Grounding determination states	7
9. When grounding must be checked	7
10. Runtime consequences	8
11. Human degradation is in scope	8
12. Local entity degradation is in scope	9
13. Sensor and proxy degradation are in scope	9
14. Commit-time revalidation	9
15. Relationship to evidence and standing	10
16. Fail-closed rules	10
17. Failure modes addressed	10
18. Canonical home	11
19. Explicit bridge	11
20. Hidden bridges	11
21. Earth paragraph	12
22. Conclusion	12

Abstract

This document defines the **Actor Grounding Layer (AGL)**.

AGL exists because a system may remain procedurally coherent while already depending on a source that is too degraded, detached, simulated, stale, overloaded, or operationally ungrounded to support runtime reliance.

Review alone does not solve this. Review can classify a dispute. Review can freeze a path. Review can bind an outcome. But review may begin **too late** if the initiating actor, source, or perceptual path should never have been allowed to move the runtime forward in the first place.

AGL therefore formalizes an upstream rule:

before a system may rely on a source strongly enough to act, escalate, or admit review,
it must determine whether that source is sufficiently grounded in **real present execution state** to count at all.

AGL is a **hard gate by default**. It is not a mood. It is not commentary. It is not post-hoc explanation.

1. Purpose

To define:

- what grounding means in operational terms,
- which actor and source classes require grounding qualification,
- when grounding must be checked,
- what runtime consequences follow from degraded or missing grounding,
- how grounding differs from evidence review,
- how grounding interacts with hardware perimeter, witness, and arbitration,
- and how a long-lived system remains fail-closed before conflict becomes expensive.

2. Scope

This document applies to source-state qualification before or during:

- action initiation,
- escalation requests,
- privileged execution,
- review admission,
- oracle / remote witness requests,
- re-entry and release,
- delegation and proxy-origin execution,
- continuity-sensitive transitions,
- and runtime situations in which the source itself may be too ungrounded to authorize normal progression.

It applies to four primary source classes:

- human / operator state,
 - local entity initiation state,
 - sensor / perception state,
 - delegated / proxy / external origin state.
-

3. Non-goals

AGL does **not**:

- replace ARL,
- replace L4 Witness,
- replace hardware perimeter controls,
- prove metaphysical authenticity,
- read minds,
- declare moral worth,
- or solve every dispute by itself.

AGL also does not guarantee truth. It does something narrower and more important:

it constrains whether a source may be relied upon **strongly enough** to move the system at all.

4. Core principle

4.1 Grounding is a precondition, not a courtesy

A source may be coherent in form and still be too ungrounded for runtime reliance.

4.2 Runtime reliance is stronger than mere visibility

A system may see a signal, store it, display it, or witness-bind it without granting it enough force to:

- open or widen privilege,
- authorize action,
- trigger escalation,
- enter review as a valid initiating basis,
- or re-enter lawful flow.

4.3 Fail-closed before elegant continuation

Where grounding is insufficient, the default is not fluid continuation. The default is narrowed authority, hold, freeze, quarantine, stronger qualification, or denial.

4.4 Re-grounding may be required at commit

Grounding is not only an intake question. A source may be grounded at initiation and drift before execution binds consequence.

4.5 A system may be formally clean and still operationally ungrounded

The log may look valid. The actor may look authorized. The signal may be properly formatted. The path may still be wrong to trust **now**.

5. Architectural position

5.1 Relation to ARL

ARL answers what happens once conflict enters procedural form:

- standing,
- evidence admissibility,
- freeze discipline,
- review,
- outcome,
- appeal,
- lawful or unlawful re-entry.

AGL sits **upstream** of that. It asks whether a source is grounded enough to support runtime reliance before ordinary progression, escalation, or procedural review even begins.

5.2 Relation to L4 Witness

L4 Witness binds what happened. AGL constrains whether the source may authorize or materially influence what is allowed to happen next.

Witness without grounding can still record a structurally elegant mistake.

5.3 Relation to L4 Hardware Perimeter

Hardware perimeter defines real operational conditions:

- network mode,
- radio state,
- power path,
- anomaly flags,
- physical access,
- maintenance state,
- circuit-breaker posture.

AGL consumes that reality as grounding input. It does not pretend runtime begins from a neutral abstract source.

5.4 Relation to ECC implementation

Implementation-facing ARL documents already contain:

- intake hooks,
- hold-entry hooks,
- privilege narrowing,
- runtime refusal,
- review routing,
- witness events,

- dispute persistence.

AGL gives the missing upstream semantic gate to those control points.

6. What AGL qualifies

AGL qualifies four source classes.

6.1 Human / operator grounding

Whether the human source is grounded enough to support reliance at the present execution boundary.

This may include:

- overload,
- fatigue,
- sleep deprivation,
- stress,
- panic,
- impairment,
- degraded attention,
- unstable privilege context,
- or other states where a human source is still visible but no longer sufficiently grounded for ordinary authority.

6.2 Local entity initiation grounding

Whether the local continuity-bearing entity is in a sufficiently grounded state to:

- initiate,
- escalate,
- authorize,
- or request external assistance.

This may include:

- unresolved drift,
- continuity fracture,
- degraded runtime state,
- witness-chain inconsistency,
- overloaded local loops,
- unresolved deadlock,
- or narrowed local confidence under L4 pressure.

6.3 Sensor / perception grounding

Whether the perceptual path itself is grounded enough to support runtime reliance.

This may include:

- stale sensor state,
- missing telemetry,

- corrupted input,
- detached interface,
- degraded hardware path,
- untrusted acquisition chain,
- or ambiguity too strong for direct runtime dependence.

6.4 Delegated / proxy / external origin grounding

Whether the source remains anchored strongly enough to real execution state rather than drifting into abstraction through:

- delegation chains,
 - proxy actors,
 - simulated surfaces,
 - replayed inputs,
 - provider-mediated indirection,
 - or remote-origin requests detached from the live operational boundary.
-

7. Grounding dimensions

Grounding is not one scalar. A source may be strong on one dimension and weak on another.

7.1 Physical grounding

Whether the source remains tied to the live physical and operational perimeter.

7.2 Temporal grounding

Whether the source is fresh enough and present enough to current execution time.

7.3 Cognitive grounding

Whether the source is sufficiently stable in attention, load, and interpretive integrity.

7.4 Perceptual grounding

Whether the signal arrives through a sufficiently trustworthy and current sensor path.

7.5 Authority grounding

Whether the source still holds lawful privilege at the moment of reliance.

7.6 Continuity grounding

Whether the source remains part of the same bounded continuity-bearing line rather than a drifted or fractured derivative.

7.7 Environmental grounding

Whether current L4 conditions still support reliance or have materially degraded the context.

8. Grounding determination states

AGL should avoid fake precision. But it must still emit bounded machine-meaningful outcomes.

Recommended grounding states:

- GROUNDED
- GROUNDED_WITH_CAUTION
- HOLD_REQUIRED
- RUNTIME_RELIANCE_DENIED
- QUARANTINE_REQUIRED
- REVALIDATE_AT_COMMIT

8.1 GROUNDED

The source is sufficiently grounded for the requested reliance under current scope.

8.2 GROUNDED_WITH_CAUTION

The source may support bounded reliance, but privilege, timing, or execution scope should be narrowed.

8.3 HOLD_REQUIRED

The source is not yet grounded enough for ordinary continuation. A bounded hold is required before progression.

8.4 RUNTIME_RELIANCE_DENIED

The source may remain visible or recordable, but it must not authorize action, escalation, or ordinary progression.

8.5 QUARANTINE_REQUIRED

The source, branch, or state path is ungrounded strongly enough that normal flow must isolate the affected material.

8.6 REVALIDATE_AT_COMMIT

The source may proceed provisionally, but grounding must be checked again when consequence actually binds.

9. When grounding must be checked

AGL checks are required at the following moments.

9.1 At intake

When a signal first attempts to affect runtime posture.

9.2 Before privileged execution

When the source attempts to widen or use authority.

9.3 Before escalation

When the source requests review, oracle help, delegation, or stronger execution path.

9.4 At commit

When action, release, or state transition is about to bind consequence.

9.5 Before re-entry

When previously held, frozen, or quarantined paths seek lawful return.

9.6 After major anomaly

When L4 signals materially change the environment, posture, or source integrity.

10. Runtime consequences

Grounding outcomes must change runtime posture. Otherwise AGL is decorative.

10.1 Proceed

Allowed only when grounding is sufficient for the requested scope.

10.2 Narrow privilege

A grounded-but-risky source may be allowed only under reduced execution rights.

10.3 Open hold

The system may permit bounded pause and qualification before progression.

10.4 Force freeze or quarantine

Where grounding failure threatens continuity, privilege, or trustworthy execution.

10.5 Deny escalation

A source may be visible but not grounded enough to request review, oracle usage, or stronger path.

10.6 Require stronger basis

The source may remain part of history while still being insufficient for runtime reliance.

11. Human degradation is in scope

AGL explicitly includes human cognitive and physiological degradation.

This is not moral judgment. It is execution reality.

A human may remain:

- the owner,
- the anchor,
- the responsible party,
- and still be too degraded **in this moment** to authorize ordinary runtime dependence without stronger qualification.

This includes cases where the system must treat the human source as:

- grounded but narrowed,

- hold-required,
- or not fit for direct high-impact reliance right now.

A serious system does not erase human agency. It protects it from becoming a bad commit under bad state.

12. Local entity degradation is in scope

AGL also applies to the local entity itself.

A persistent entity may be:

- continuity-bearing,
- memory-rich,
- and still too drifted, overloaded, fractured, or environmentally degraded to authorize normal progression.

The system must not assume that local persistence alone equals current grounding.

Continuity matters. Present-state integrity matters too.

13. Sensor and proxy degradation are in scope

AGL must resist a recurring modern mistake: assuming that because a signal is available, it is operationally fit for reliance.

That mistake becomes severe under:

- stale sensors,
- detached interfaces,
- provider proxies,
- replay surfaces,
- simulation loops,
- and delegated chains where responsibility becomes thinner at every hop.

Availability is not grounding. Visibility is not reliance.

14. Commit-time revalidation

A source may be sufficiently grounded at intake and insufficiently grounded later. Therefore, high-impact paths should support commit-time revalidation of grounding, especially when:

- delay is long,
- environmental drift is high,
- privilege is strong,
- or consequence is difficult to reverse.

This is not indecision. It is preventing stale legitimacy from silently becoming live action.

15. Relationship to evidence and standing

AGL must remain distinct from two nearby questions.

15.1 AGL is not evidence admissibility

Evidence asks: - is this record fit to enter review?

AGL asks: - is this source fit to support runtime reliance strongly enough to move the system?

15.2 AGL is not standing

Standing asks: - who may initiate, support, or sustain a dispute?

AGL asks: - is the source grounded enough **now** for that procedural right to matter operationally?

Standing without grounding may still be too late. Grounding without standing may still be insufficient. They are adjacent, not identical.

16. Fail-closed rules

16.1 No grounding, no ordinary progression

If grounding is insufficient, ordinary continuation must not remain the default.

16.2 No stale legitimacy

Authority established earlier does not automatically remain grounded now.

16.3 No abstraction laundering

Delegation, proxying, or simulated cleanliness must not conceal loss of present-state grounding.

16.4 No silent commit under degraded source state

A system must not bind high-impact consequence while the source is known to be materially ungrounded.

16.5 No smoothing of unresolved grounding

Weak grounding should not be cosmetically converted into acceptable runtime dependence through confident phrasing.

17. Failure modes addressed

AGL is designed to reduce the following failures:

- procedurally valid but operationally ungrounded initiation,
- stale authority becoming live execution,
- degraded human/operator state passing as ordinary authorization,
- sensor availability mistaken for perceptual trustworthiness,
- proxy or delegated abstraction replacing real present execution state,
- review beginning too late to prevent bad reliance,

- and elegant downstream reasoning built on an already ungrounded start.
-

18. Canonical home

AGL belongs canonically in ester-reality-bound because the layer it addresses is:

- operational,
- physical,
- present-state-sensitive,
- anomaly-aware,
- and tightly coupled to L4 conditions.

ARL consumes grounded dispute conditions. L4 Witness records what happened. Hardware perimeter constrains where execution is even sane.

AGL sits between those as the explicit question:

is the source grounded enough for runtime reliance **before** we let ordinary procedural machinery continue?

19. Explicit bridge

L4 Hardware Perimeter <-> Actor Grounding Layer <-> ARL <-> L4 Witness

Perimeter defines live operational conditions. AGL qualifies source-state grounding under those conditions. ARL governs bounded conflict once admitted. Witness preserves what actually happened.

20. Hidden bridges

20.1 Hidden bridge #1 — ERB <-> SER

AGL explains why pre-admissibility hold is not merely a review convenience. It is the point where grounding failure can stop the path before ARL is forced to clean up a bad initiation.

20.2 Hidden bridge #2 — ERB <-> ECC

AGL gives upstream meaning to implementation-facing concepts such as:

- intake hooks,
- hold-entry hooks,
- privilege narrowing,
- runtime refusal,
- commit-time revalidation,
- and re-entry legality.

Without grounding discipline, those hooks remain mechanically useful but semantically incomplete.

21. Earth paragraph

In a real operating room, no serious team says: “the chart is complete, the checklist is signed, and the room looks calm — proceed,” without also asking whether the hand is steady, the patient is oxygenated, the monitors are live, the instruments are trustworthy, and the room is still inside safe conditions.

If the source state is wrong, you stop **before** the cut. Not after the incision, not after the audit, not after the review committee.

AGL is that logic for long-lived digital systems.

22. Conclusion

Actor Grounding Layer v0.1 formalizes a simple but structural rule:

before a system may rely on a source strongly enough to act, escalate, or admit review,
it must determine whether that source is sufficiently grounded in real present execution state to count at all.

This is not an alternative to ARL. It is the upstream condition that keeps ARL from becoming too late.

It is not an alternative to witness. It keeps witness from recording a beautifully structured mistake.

And it is not an alternative to perimeter. It gives perimeter a runtime semantic consequence.

Grounding is not only something to assess. For serious systems, grounding may be something the system must require **before action exists at all**.